Politecnico di Torino

Explainable and Trustworthy AI

Exam Questions

Students:
Stefano Sammartino
Marco Roberto

Professor:
Prof. Eliana Pastor

Academic Year
2025-2026

Contents

1 Trustworthy AI: Motivation and Definitions **4**
1.1 Accountability in Trustworthy AI — May 2025 (Sample) · 1 pt · MCQ . 4

2 Explainable AI: Taxonomy **6**
2.1 Model-Agnostic Explainability — June 2025 (Exam) · 1 pt · MCQ . . . 6
2.2 Local Interpretability — May 2025 (Sample) · 1 pt · MCQ 8

3 Pre-Modeling Explainability **10**

4 In-Modeling Explainability **11**

5 Post-Modeling Explainability **12**
5.1 Permutation Feature Importance — June 2026 (Exam) · 2 pts · Open . 12
5.2 Surrogate Models — May 2025 (Sample) · 3 pts · Open 15
5.3 LIME — The Complexity Term $\Omega(g)$ — May 2026 (Interactive Questions) · 1 pt · MCQ 17
5.4 LIME — The Core Trade-off — May 2026 (Interactive Questions) · 1 pt · MCQ 19
5.5 LIME — Order of the High-Level Steps — May 2026 (Interactive Questions) · 1 pt · MCQ 21
5.6 LIME — Interpretable Representation for Images — May 2026 (Interactive Questions) · 1 pt · MCQ 23
5.7 LIME — Unrealistic Neighbor Samples — May 2026 (Interactive Questions) · 1 pt · MCQ 25
5.8 LIME — Reducing Explanation Instability — May 2026 (Interactive Questions) · 1 pt · MCQ 27
5.9 Explaining a Credit-Scoring Model with LIME — May 2025 (Sample) · 4 pts · Open 29
5.10 Explaining by Removing — Interpreting a PredDiff Score — May 2026 (Interactive Questions) · 1 pt · MCQ 31
5.11 Explaining by Removing — A Limitation Shapley Values Address — May 2026 (Interactive Questions) · 1 pt · MCQ 33
5.12 Shapley Values — The Marginal Contribution Term — May 2026 (Interactive Questions) · 1 pt · MCQ 35
5.13 Shapley Values — Number of Coalitions — May 2026 (Interactive Questions) · 1 pt · MCQ 37

5.14 Shapley Values — The Null Player Axiom — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 39
5.15 SHAP and Shapley Values — June 2025 (Exam) · 2.5 pts · Open . . . 41
5.16 Gradient — What the Gradients Are Computed With Respect To — June 2026 (Exam) · 1 pt · MCQ . . . 43
5.17 Gradient — Key Idea of Integrated Gradients — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 45
5.18 Gradient — Main Limitation of Grad-CAM — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 47
5.19 Gradient — Grad-CAM vs. Vanilla Gradient — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 49
5.20 Gradient — The Sensitivity Axiom — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 51
5.21 Explaining a Black-Box Healthcare System (Tabular Data) — May 2025 (Sample) · 7 pts · Open . . . 53
5.22 Explaining a Black-Box Medical-Imaging Diagnosis System — June 2025 (Exam) · 6 pts · Open . . . 57
5.23 Explaining a Fraud-Detection System (Tabular Data) — June 2026 (Exam) · 5 pts · Open . . . 60

6 Evaluating Explainability 64
6.1 Faithfulness vs. Plausibility — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 64
6.2 Faithfulness vs. Plausibility — Can an Explanation Be Plausible but Not Faithful? — June 2026 (Exam) · 1.5 pts · Open . . . 66
6.3 Removal-based Evaluation of Top-k Features — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 68
6.4 Capturing Completeness (Sufficiency) — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 70
6.5 Detecting an Unfaithful Explanation — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 72
6.6 Testing Contrastivity (Target Sensitivity) — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 74

7 Concept-based Explainable AI 76
7.1 Limitations of Saliency Maps and the Concept-based Remedy — June 2025 (Exam) · 3 pts · Open . . . 76
7.2 Explainable-by-Design with Concepts but No Concept Labels — June 2026 (Exam) · 3 pts · Open . . . 78

8 Introduction to Natural Language Processing 81

9 Counterfactual Explanations 82
9.1 Counterfactual Explanations for Model Predictions — May 2025 (Sample) · 4 pts · Open . . . 82
9.2 Why Counterfactuals Are Example-Based — May 2026 (Interactive Questions) · 1 pt · MCQ . . . 84

9.3 Property Ensuring Minimal Change — May 2026 (Interactive Questions) · 1 pt · MCQ 86
9.4 Meaning of Sparsity — May 2026 (Interactive Questions) · 1 pt · MCQ . 88
9.5 Least Actionable Counterfactual — June 2026 (Exam) · 1 pt · MCQ . . 90
9.6 Goal of the Loss in Wachter et al. — May 2026 (Interactive Questions) · 1 pt · MCQ 92
9.7 How DiCE Improves Counterfactual Generation — May 2026 (Interactive Questions) · 1 pt · MCQ 94

10 Attention-based Explainability **96**

11 Mechanistic Interpretability **97**
11.1 False Statements about Mechanistic Interpretability — June 2025 (Exam) · 1 pt · MCQ 97

12 Adversarial Attacks and Defenses **100**
12.1 Adversarial Attacks and Their Importance — June 2025 (Exam) · 2.5 pts · Open 100
12.2 Adversarial Attacks in Trustworthy AI — Are They a Real Threat? — June 2026 (Exam) · 2.5 pts · Open 103

Chapter 1

Trustworthy AI: Motivation and Definitions

1.1 Accountability in Trustworthy AI — May 2025 (Sample)

· 1 pt · MCQ

Task

Which of the following best describes **Accountability** in the context of Trustworthy AI?

- ☐ The requirement that AI systems operate without any human oversight.
- ☐ The responsibility of the entities involved to ensure and answer for the outcomes produced by AI systems.
- ☐ The ability of AI systems to self-correct when errors occur.
- ☐ The focus on minimizing the cost of deploying AI systems.

Solution

Solution

The responsibility of the entities involved to ensure and answer for the outcomes produced by AI systems.

Accountability means that the entities involved in the AI lifecycle must be **responsible** for their actions and for the consequences produced by the AI system, and must be able to answer for those outcomes.

Analysis of Options

- *The requirement that AI systems operate without any human oversight.* — **Wrong:**
  Removing human oversight contradicts the requirement of *Human Agency and Oversight* (e.g. Human-in-the-loop, Human-on-the-loop, Human-in-command). Accountability does not imply autonomy from humans.
- *The ability of AI systems to self-correct when errors occur.* — **Wrong:**
  Self-correction relates to *Technical Robustness and Safety*, not Accountability.
- *The focus on minimizing the cost of deploying AI systems.* — **Wrong:**
  Cost minimization relates to *Societal and Enviromental Well-being*, not Accountability.

Theory Reference

Chapter – Trustworthy AI: Motivation and Definitions, Section **Key Requirements for Trustworthy AI** → Subsection **4. Accountability**:

Entities must be responsible for their actions and the consequences of AI. Accountability rests on two pillars:

- **Responsibility**: shared among **AI Users** (ensuring appropriate use), **Businesses** (establishing guidelines), **Developers** (building safety measures) and **Data Providers** (ensuring data quality).
- **Auditability**: the assessment of algorithms and processes through internal and external auditors.

Chapter 2

Explainable AI: Taxonomy

2.1 Model-Agnostic Explainability — June 2025 (Exam) · 1 pt · MCQ

Task

What does **"Model-Agnostic"** mean in the context of Explainability?

- ☐ The explanation method is specific to a particular Model type.
- ☐ The explanation method can be applied to any ML Model.
- ☐ The explanation method ignores the input data.
- ☐ The explanation method is only applicable to Neural Networks.

Solution

Solution

The explanation method can be applied to any ML Model.

A **Model-Agnostic** method treats the Model as a **black-box** and can be applied to **any** ML Model, regardless of its internal structure, relying only on altering inputs and observing the output predictions/probabilities.

Analysis of Options

- *The explanation method is specific to a particular Model type.* — **Wrong:** A method tied to a specific Model type is *Model-dependent* (Model-specific), the opposite of Model-Agnostic. These rely on the internal mathematical structure of the Model (e.g. Gradient-based approaches for Neural Networks or Split-Analysis for Random Forests).
- *The explanation method ignores the input data.* — **Wrong:** Model-Agnostic methods do *not* ignore the input data; they explicitly alter inputs and observe outputs (e.g. LIME, SHAP).
- *The explanation method is only applicable to Neural Networks.* — **Wrong:** Being limited to Neural Networks would make a method Model-dependent, not Agnostic.

Theory Reference

Chapter – Explainable AI: Taxonomy, Section **Generalizability of Explainability** → Subsection **Advantages of Model-Agnostic Solutions**:

Model-Agnostic Solutions are applicable to **any** Model: they treat the Model as a black-box, relying only on altering inputs and observing the output predictions/probabilities.
Their advantages include Model flexibility and compatibility, explanation flexibility, representation flexibility, lower cost to switch the underlying Model, and easier Model comparison.

2.2 Local Interpretability — May 2025 (Sample) · 1 pt · MCQ

Task

What does **Local Intepretability** in Explainable Artificial Intelligence (XAI) refer to?

- ☐ Understanding the overall behavior of the Model.
- ☐ Understanding individual predictions or decisions made by the Model.
- ☐ Understanding the data preprocessing steps in the Model.
- ☐ Understanding the training process of the Model.

Solution

Solution

Understanding individual predictions or decisions made by the Model.

Local (individual) Intepretability explains the reasons behind a **single, specific prediction** — e.g. "Why did the Model make *this* decision for *this* patient?" — as opposed to the Global behavior of the Model.

Analysis of Options

- *Understanding the overall behavior of the Model.* — **Wrong:**
  Understanding the overall behavior across the entire dataset is the definition of *Global* explanations.
- *Understanding the data preprocessing steps in the Model.* — **Wrong:**
  Data preprocessing belongs to the Pre-Modeling / data-exploration stage, not to the scope of Intepretability.
- *Understanding the training process of the Model.* — **Wrong:**
  Understanding the training process is unrelated to the scope (Local vs. Global) of an explanation.

Theory Reference

Chapter – Explainable AI: Taxonomy, Section **Scope of Explainability**:

Individual/Local Explanations explain the reasons behind a single and specific prediction (e.g. "Why did the Model make *this* decision for *this* patient?"), presenting some kind of textual or visual artifact easily understandable for a human together with the Model's prediction.
This contrasts with Global Explanations (overall behavior across the entire dataset) and Subgroup Explanations (behavior within specific clusters or data subgroups).

Chapter 3

Pre-Modeling Explainability

Unfortunately, no questions published for this Chapter until now!

Chapter 4

In-Modeling Explainability

Unfortunately, no questions published for this Chapter until now!

Chapter 5

Post-Modeling Explainability

5.1 Permutation Feature Importance — June 2026 (Exam) · 2 pts · Open

Task

Explain **permutation feature importance**. Describe the main idea behind the method and how the resulting explanations should be interpreted. Specify whether the explanation is local or global.

Solution

Solution

Permutation Feature Importance (PFI) is a **post-hoc**, **model-agnostic**, **global** explainability method (it measures how much the model *as a whole* relies on each feature).
The main idea: to assess how important a feature is, **randomly shuffle (permute) that feature's values** across the whole dataset — breaking its relationship with the target — and measure how much the model's performance *drops* compared to the original (unshuffled) performance.

Main Idea (Procedure)

1. Compute a baseline performance score of the trained model on the (validation) data (e.g. accuracy, AUC, MSE).
2. Pick a feature and **permute its column**, shuffling its values across all instances so it becomes uninformative while keeping the marginal distribution.
3. Re-evaluate the model on this modified dataset and record the new performance.
4. The **importance** of the feature is the **drop in performance** (baseline − permuted). Repeat for every feature; results are typically shown as a ranked bar plot.

How to Interpret the Explanation

- **Large performance drop** ⇒ the feature is **important**: the model relies heavily on it, so destroying its signal hurts predictions.
- **Little/no drop** ⇒ the feature is **unimportant**: the model barely uses it.
- It is **global**, because it characterizes the model's *overall* reliance on each feature across the whole dataset, not the reasoning behind a single instance's prediction.

Common Mistake

Be careful with the well-known **limitations** (frequently expected in the answer):

- **Correlated features:** permuting one of two correlated features can be compensated by the other, *underestimating* importance; it can also create **unrealistic / off-manifold** samples after the shuffle.
- **Randomness / variance:** because the permutation is random, the score varies between runs; it should be **averaged over several permutations**.
- PFI is a **post-hoc** method applied to an *already-trained* model — it is *not* an in-modeling / explainable-by-design technique.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Permutation Feature Importance (PFI)**:

The importance of a feature is the increase in the model's prediction error after permuting that feature's values, breaking its link with the true outcome; a large error increase means the model depends strongly on it. It is a global, model-agnostic method, sensitive to feature correlations and to the randomness of the permutation.

5.2 Surrogate Models — May 2025 (Sample) · 3 pts · Open

Task

Outline the concept of **surrogate models** in XAI. How can surrogate models be used to interpret machine learning models?

Solution

Solution

A **surrogate model** is an *interpretable* model g (e.g. a decision tree, decision rules, or a linear/logistic model) trained to approximate the black-box prediction function f.
It is trained on data points **labeled by the black box** (i.e. using f's predictions as targets, not the ground-truth labels), so that we can read off explanations from the transparent surrogate instead of the opaque original.

Key Properties

- **Post-hoc** explainability (the black box is already trained).
- **Model-agnostic** (only input–output behavior is used, both for the model to explain and for the choice of surrogate family).
- The surrogate must closely **mimic** (be *faithful* to) the behavior of f; fidelity is measured with metrics such as MSE, Accuracy, or AUC-ROC.

Two Scopes

- **Global surrogate:** trained to explain the whole model, using a dataset (the original training set or a newly generated one from the same distribution) labeled by f (e.g. TREPAN, which extracts a decision tree from a neural network using a best-first, fidelity-guided expansion).
- **Local surrogate:** trained to explain an individual prediction. Data points are sampled or generated (e.g. via perturbation or genetic algorithms) in the *locality* of the instance to explain (e.g. LIME, LORE).

Common Mistake

Surrogate models are most commonly used for *tabular data*, because interpretable models are typically designed for and work better on this data type, or otherwise require an interpretable representation of the input.

Theory Reference

Chapter – Post-Modeling Explainability, Sections **Interpretable Global Surrogate Models** (Building steps, TREPAN, Advantages/Limitations) and **Local Surrogate Interpretable Models**.

5.3 LIME — The Complexity Term $\Omega(g)$ — May 2026

(Interactive Questions) · 1 pt · MCQ

Task

In the LIME objective — explanation$(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$ — what does $\Omega(g)$ represent and why is it minimized?

- ☐ The proximity between x and perturbed samples — minimized to focus on nearby points.
- ☐ The prediction error of the original model f — minimized to improve accuracy.
- ☐ The complexity of the surrogate model g — minimized to keep explanations interpretable.
- ☐ The number of perturbed samples generated — minimized to reduce computation time.

Solution

Solution

The complexity of the surrogate model g — minimized to keep explanations interpretable.

$\Omega(g)$ is the **complexity penalty** on the surrogate (e.g. the number of non-zero weights in a linear model, or the depth of a tree).
Minimizing it keeps g simple enough for a human to understand; shorter explanations equal higher interpretability.

Analysis of Options

- *The proximity between x and perturbed samples.* — **Wrong:**
  Proximity is captured by the weighting kernel π_x inside the loss term $\mathcal{L}$, not by Ω.
- *The prediction error of the original model f.* — **Wrong:**
  f is the fixed black box being explained; LIME does not retrain or improve its accuracy.
- *The number of perturbed samples generated.* — **Wrong:**
  The sample count is a sampling hyperparameter, not a term in the objective function.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*Mathematical Definition of LIME*):

The objective balances a local fidelity loss $\mathcal{L}(f, g, \pi_x)$ against a complexity penalty $\Omega(g)$ (e.g. number of non-zero weights or tree depth) that keeps the surrogate interpretable.

5.4 LIME — The Core Trade-off — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Which of the following best describes the trade-off captured by the full LIME objective $\mathcal{L}(f, g, \pi_x) + \Omega(g)$?

- ☐ The trade-off between model accuracy on training data and generalization to unseen data.
- ☐ The trade-off between faithfully approximating the black-box locally and keeping the surrogate model simple enough to interpret.
- ☐ The trade-off between the size of the neighborhood and the speed of computation.
- ☐ The trade-off between using interpretable features for explanation and raw features for training.

Solution

Solution

The trade-off between faithfully approximating the black-box locally and keeping the surrogate model simple enough to interpret.

The loss term $\mathcal{L}(f, g, \pi_x)$ rewards **local fidelity** to the black box, while the penalty $\Omega(g)$ rewards **simplicity** of the surrogate.
LIME jointly optimizes both, balancing faithfulness against interpretability.

Analysis of Options

- *Accuracy vs. generalization.* — **Wrong:**
  This is the classic bias–variance trade-off in ML training, not what LIME's local objective expresses.
- *Neighborhood size vs. speed.* — **Wrong:**
  That is a sampling/computational concern, not a term in the objective.
- *Interpretable features vs. raw features.* — **Wrong:**
  LIME always explains in the interpretable representation; there is no such trade in the objective.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*Mathematical Definition of LIME* and *Properties of LIME Explanations*: Interpretable, Locally Faithful):

The objective trades local faithfulness $\mathcal{L}(f, g, \pi_x)$ against surrogate complexity $\Omega(g)$ to produce an explanation that is both accurate locally and interpretable.

5.5 LIME — Order of the High-Level Steps — May 2026

(Interactive Questions) · 1 pt · MCQ

Task

In **LIME**, what is the correct order of the high-level steps?

- ☐ Generate neighborhood → Get predictions → Weight by proximity → Train interpretable model → Explain.
- ☐ Train model → Generate neighborhood → Weight by proximity → Get predictions → Explain.
- ☐ Get predictions → Generate neighborhood → Train interpretable model → Weight by proximity → Explain.
- ☐ Generate neighborhood → Train interpretable model on original labels → Weight by proximity → Explain.

Solution

Solution

Generate neighborhood → Get predictions → Weight by proximity → Train interpretable model → Explain.

LIME first **generates the neighborhood** of the instance by perturbation, **labels** those points with the black box, **weights** them by proximity to the instance, **trains the interpretable surrogate** on the weighted labeled neighborhood, and finally **explains** by reading the surrogate's weights.

Analysis of Options

- *Train model → Generate neighborhood → ...* — **Wrong:** The surrogate cannot be trained before the neighborhood exists and is labeled; the surrogate is the *last* modeling step, not the first.
- *Get predictions → Generate neighborhood → ...* — **Wrong:** Predictions are obtained *on* the generated neighbors, so the neighborhood must be created first.
- *Generate neighborhood → Train on original labels → ...* — **Wrong:** The surrogate must be trained on the *black-box labels* of the neighbors, not on original labels, and weighting precedes training.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*High-Level Steps of LIME*):

Generate the neighborhood by perturbation, label the points with the model to explain, weight them by proximity to the instance, train a weighted local interpretable model, and explain via its weights.

5.6 LIME — Interpretable Representation for Images

— May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What is an **"interpretable representation"** in LIME for images?

- ☐ Superpixel/patch segments encoded as a binary vector.
- ☐ The gradient map of the image.
- ☐ The raw pixel matrix ($W \times H \times C$).
- ☐ A learned embedding from the neural network.

Solution

Solution

Superpixel/patch segments encoded as a binary vector.

For images, LIME groups the raw pixels into **super-pixels** (contiguous patches) and represents the instance as a **binary vector** indicating the presence (1) or absence (0) of each patch.
This human-understandable representation is what perturbations toggle on and off (e.g. hiding a patch by turning it gray/black).

Analysis of Options

- *The gradient map of the image.* — **Wrong:**
  Gradient maps belong to gradient-based attribution methods, not to LIME's interpretable representation.
- *The raw pixel matrix ($W \times H \times C$).* — **Wrong:**
  Raw pixels are the *model's* representation, not interpretable; LIME explicitly abstracts them into super-pixels.
- *A learned embedding from the neural network.* — **Wrong:**
  A latent embedding is not human-understandable and is model-specific, contradicting LIME's model-agnostic, interpretable design.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*The Three Pillars of LIME*: Interpretable Data Representation):

LIME maps the raw input to an interpretable representation — super-pixels for images, Bag-of-Words for text — encoded as a binary presence/absence vector. Tabular data is usually already interpretable.

5.7 LIME — Unrealistic Neighbor Samples — May 2026

(Interactive Questions) · 1 pt · MCQ

Task

Why might LIME produce unrealistic neighbor samples, and what is the main reason for this?

- ☐ Because the linear surrogate model is too simple to capture complex boundaries.
- ☐ Because perturbations are generated independently per feature, ignoring correlations between features.
- ☐ Because proximity is measured using cosine similarity instead of Euclidean distance.
- ☐ Because the number of neighbors sampled is too small.

Solution

Solution

Because perturbations are generated independently per feature, ignoring correlations between features.

The random perturbation process perturbs features **independently**, so it can create combinations that never occur in the real data distribution (e.g. a person 1.8m tall who weighs 20kg).
These unrealistic / off-manifold samples can mislead the local surrogate.

Analysis of Options

- *The linear surrogate is too simple.* — **Wrong:**
  Surrogate simplicity affects *local fidelity*, not whether the sampled neighbors are realistic.
- *Cosine vs. Euclidean proximity.* — **Wrong:**
  The choice of distance metric changes weighting, not the realism of the generated points themselves.
- *Too few neighbors sampled.* — **Wrong:**
  A small sample causes *instability*, but realism is governed by *how* points are generated, not how many.

Common Mistake

Independent perturbation that ignores feature correlations is the root cause of LIME's unrealistic (off-manifold) neighbors. Methods that respect the data manifold or feature correlations (e.g. LACE, which uses the actual surrounding data) mitigate this issue.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*Limitations of LIME*: Unrealistic Samples):

The random perturbation may generate synthetic neighbors that are entirely unrealistic because it does not consider correlations between features.

5.8 LIME — Reducing Explanation Instability — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

A data scientist runs LIME twice on the same instance and gets different explanations. Which of the following actions would most directly reduce this problem?

- ☐ Switch from a linear surrogate to a decision tree.
- ☐ Increase the number of perturbed samples generated for the neighborhood.
- ☐ Use a smaller value of K (number of interpretable features).
- ☐ Replace cosine similarity with Euclidean distance as the proximity measure.

Solution

Solution

Increase the number of perturbed samples generated for the neighborhood.

LIME's instability comes from the fact that the neighborhood is built by **random sampling**, so repeated runs on the same instance can yield slightly different explanations.
Generating more perturbed samples reduces the variance of the fitted surrogate, making repeated explanations more consistent.

Analysis of Options

- *Switch to a decision tree.* — **Wrong:**
  Changing the surrogate family does not address the randomness of the sampled neighborhood, which is the source of the variation.
- *Use a smaller K.* — **Wrong:**
  Fewer interpretable features yields a sparser explanation but does not stabilize the underlying sampling.
- *Cosine vs. Euclidean distance.* — **Wrong:**
  The proximity metric reshapes weights but does not reduce the run-to-run sampling variance.

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*Limitations of LIME*: Instability):

Because the neighborhood is generated via random sampling, running LIME multiple times on the same instance can yield slightly different explanations; using more samples reduces this instability.

5.9 Explaining a Credit-Scoring Model with LIME

— May 2025 (Sample) · 4 pts · Open

Task

You are provided with a black-box credit-scoring model. Design an approach using **LIME** to explain why a particular individual was denied credit. Detail the steps you would take and the type of explanations you would generate.

Solution

Solution

LIME (Local Interpretable Model-agnostic Explanations) is a **post-hoc**, **model-agnostic**, **local** surrogate method.
It explains the individual "denied" prediction by fitting a simple interpretable (linear) model in the neighborhood of that applicant.

High-level Steps of LIME (given the instance x to explain)

1. **Generate the neighborhood** of x by perturbing it, creating synthetic neighboring data points. For tabular data, numerical features are perturbed by sampling from $\mathcal{N}(0, 1)$ and rescaling with the training statistics; categorical features are sampled according to the training distribution.

2. **Label** the generated local points using the black-box model to explain.

3. **Weight** the points by their proximity to the instance x (closer points get higher weight).

4. **Train a weighted local interpretable model** (a linear model, optionally with LASSO/Ridge to limit the number of features K) on these labeled points.

5. **Explain** by reading the surrogate's coefficients.

Type of Explanation Generated

- **Feature attribution:** an importance score (weight) for each interpretable feature.

- For tabular scoring data — already an interpretable representation — this yields importance scores per feature (e.g. income, debt, credit history), naturally presented as a **bar plot** showing which features pushed the decision toward "denied".

Theory Reference

Chapter – Post-Modeling Explainability, Section **LIME** (*High-Level Steps of LIME*, *The Three Pillars of LIME*, including tabular perturbation and the complexity parameter K).

5.10 Explaining by Removing — Interpreting a PredDiff Score — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

A **PredDiff** score for an attribute A on instance x is -0.15. What is the correct interpretation?

- ☐ Attribute A has no influence on the prediction.
- ☐ Attribute A pushes the prediction against class c by 0.15.
- ☐ Attribute A pushes the prediction toward class c by 0.15.
- ☐ The model is 15% less accurate when A is removed.
- ☐ The model is 15% more accurate when A is removed.

Solution

Solution

Attribute A pushes the prediction against class c by 0.15.

PredDiff measures how the predicted probability of class c changes when attribute A is present versus removed.
A **negative** score means A provides a **negative contribution** — it pushes the model *against* the prediction for class c; the magnitude 0.15 indicates the strength of that effect.

Analysis of Options

- *A has no influence.* — **Wrong:**
  A non-zero score (-0.15) means A does affect the prediction.
- *A pushes toward class c by 0.15.* — **Wrong:**
  A *positive* contribution pushes toward class c; the sign here is negative.
- *The model is 15% less/more accurate when A is removed.* — **Wrong:**
  PredDiff quantifies a change in the *prediction for this instance*, not a change in the model's overall accuracy.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Prediction Difference (PredDiff)** (*Interpreting PredDiff*: magnitude, positive vs. negative contribution):

PredDiff attributes to a feature the change in the predicted class probability when that feature is removed; a positive value pushes toward class c, a negative value pushes against it, and the magnitude reflects the impact.

5.11 Explaining by Removing — A Limitation Shapley Values Address — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Which of the following is a limitation of PredDiff that **Shapley values** are specifically designed to address?

- ☐ PredDiff cannot produce feature attributions.
- ☐ PredDiff does not consider interactions between features when computing individual attributions.
- ☐ PredDiff requires a differentiable model.
- ☐ PredDiff only works for binary classification.

Solution

Solution

PredDiff does not consider interactions between features when computing individual attributions.

PredDiff removes one feature at a time and ignores how features **interact**. Shapley values evaluate a feature's marginal contribution over **all possible coalitions** of the other features, explicitly accounting for feature interactions.

Analysis of Options

- *PredDiff cannot produce feature attributions.* — **Wrong:** Producing per-feature attributions is exactly what PredDiff does.
- *PredDiff requires a differentiable model.* — **Wrong:** PredDiff is removal-based and model-agnostic; it needs no gradients.
- *PredDiff only works for binary classification.* — **Wrong:** It applies to multi-class outputs as well; this is not its limitation.

Theory Reference

Chapter – Post-Modeling Explainability, Sections **PredDiff** (*Limitations*: "does not consider interactions between features") and **Shapley Values for XAI** (*Considers Interactions*):

Unlike single-feature removal, the Shapley value evaluates a feature's marginal contribution across all coalitions, capturing interactions between features.

5.12 Shapley Values — The Marginal Contribution Term — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

In the Shapley value formula, what does the term $v(S) - v(S \setminus \{i\})$ represent?

- ☐ The total prediction for coalition S.
- ☐ The marginal contribution of player i to coalition S.
- ☐ The average prediction across all coalitions.
- ☐ The penalty for adding player i to an already complete coalition.

Solution

Solution

The marginal contribution of player i to coalition S.

$v(S)$ is the payoff of coalition S and $v(S\backslash\{i\})$ is the payoff of the same coalition *without* player i.
Their difference is exactly how much i **adds** to that coalition — its marginal contribution. The full Shapley value is the weighted average of this term over all coalitions containing i.

Analysis of Options

- *The total prediction for coalition S.* — **Wrong:**
  That is just $v(S)$, not the difference.
- *The average prediction across all coalitions.* — **Wrong:**
  The averaging over coalitions happens *outside* this term, via the weighted sum in the full Shapley formula.
- *The penalty for adding i to a complete coalition.* — **Wrong:**
  There is no penalty concept here; the term is a contribution, and S need not be complete.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Shapley Values for XAI** (*Game Theory Origins*, the Shapley Value formula):

The Shapley value averages the marginal contribution $v(S) - v(S \setminus \{i\})$ of player i over all coalitions S that contain it, weighted by $\frac{(|N|-|S|)!\,(|S|-1)!}{|N|!}$.

5.13 Shapley Values — Number of Coalitions — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

For a model with 20 features, how many *baseline* coalitions of the other features must be evaluated to compute the *exact* Shapley value for a **single** feature (i.e. the coalitions compared with vs. without that feature)?

- ☐ 20.
- ☐ 400.
- ☐ $2^{19} = 524{,}288$.
- ☐ $20! \approx 2.4 \times 10^{18}$.

Solution

Solution

$2^{19} = 524{,}288$.

To evaluate the marginal contribution of one target feature, each of the *remaining* $n-1$ features is either present or absent, giving 2^{n-1} baseline coalitions to compare "without" vs. "with" the target feature.
For $n = 20$ this is $2^{19} = 524{,}288$. (Equivalently, the theory's 4-feature example evaluates $2^3 = 8$ baseline coalitions for one feature.) This exponential growth — and the 2^n cost of the *full* attribution over all features — is what motivates approximations such as KernelSHAP and Monte-Carlo sampling.

Analysis of Options

- 20. — **Wrong:**
  This counts features, not subsets of them.
- 400. — **Wrong:**
  $20^2 = 400$ has no combinatorial meaning here.
- $20! \approx 2.4 \times 10^{18}$. — **Wrong:**
  20! counts *orderings* (permutations); the Shapley computation enumerates *subsets*.

Common Mistake

The number of coalitions grows exponentially: 2^{n-1} baseline coalitions per feature, and 2^n overall. This makes exact Shapley values intractable for large n, which is precisely why approximate methods such as KernelSHAP and sampling-based (Monte-Carlo) estimators exist.

Theory Reference

Chapter – Post-Modeling Explainability, Sections **Shapley Values for XAI** (*Example: Coalitions to Evaluate* — "$2^3 = 8$ distinct baseline coalitions" for one feature) and **Shapley Approximation** (the 2^N exponential cost of the full computation).

5.14 Shapley Values — The Null Player Axiom –

May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Feature B never changes the model output regardless of which coalition it joins. According to the **Null Player** axiom, what Shapley value does it receive, and which real-world scenario does this best reflect?

- ☐ $\varphi_B = 1.0$; a feature that perfectly predicts the target.
- ☐ $\varphi_B = 0$; a completely irrelevant feature that the model ignores.
- ☐ $\varphi_B = \mathbb{E}[f(X)]$; a feature equal to the average prediction.
- ☐ $\varphi_B = -1.0$; a feature that actively harms prediction.

Solution

Solution

$\varphi_B = 0$**; a completely irrelevant feature that the model ignores.**

The **Null Player** axiom states that a player who adds absolutely zero marginal value to *every* possible coalition receives a final contribution of exactly 0.
This reflects a feature the model simply does not use.

Analysis of Options

- $\varphi_B = 1.0$*; perfect predictor.* — **Wrong:**
  A perfect predictor would have a *large* contribution, the opposite of a null player.
- $\varphi_B = \mathbb{E}[f(X)]$. — **Wrong:**
  The average prediction is the *baseline* (ϕ_0) of the explanation, not an individual feature's attribution.
- $\varphi_B = -1.0$*; actively harms.* — **Wrong:**
  A harmful feature would have a non-zero (negative) contribution; a null player contributes nothing in *any* coalition.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Shapley Values for XAI** (*Properties of the Shapley Value*: Efficiency, Symmetry, Linearity, Null Player):

The Null Player axiom assigns a Shapley value of zero to any feature whose marginal contribution is zero across all coalitions.

5.15 SHAP and Shapley Values — June 2025 (Exam) · 2.5 pts · Open

Task

Briefly describe the core idea behind **SHAP** (SHapley Additive exPlanations) and how it relates to Shapley values from cooperative game theory. Mention which kind of explanation representation it provides to the user. Outline one advantage of using SHAP for model interpretability.

Solution

Solution

SHAP is a **post-hoc, model-agnostic, local** explanation method built directly on Shapley values. It explains a prediction by treating the features as *players* in a cooperative game whose *payout* is the model prediction relative to the average.
Each feature receives a **Shapley value** ϕ_i: its fair contribution to the difference between the actual prediction and the average (baseline) prediction $E[f(X)]$, averaged over all possible coalitions/orderings of features.

Relation to Game Theory

The Shapley value, originally from cooperative game theory, is the unique attribution satisfying *efficiency*, *symmetry*, *linearity* and *null player*.
SHAP imports these guarantees into XAI as an **additive feature attribution** model $g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z'_i$, where $z' \in \{0,1\}^M$ is the interpretable (present/absent) representation and $\phi_0 = E[f(X)]$. It is proven to be the only such additive model satisfying *local accuracy*, *missingness* and *consistency*.

Explanation Representation

Feature attribution — an importance (contribution) score per feature, typically shown locally as force/bar plots, and aggregated (e.g. mean absolute Shapley value, summary plots, dependence plots) for global insights.

One Advantage

It enjoys a *solid theoretical foundation* with desirable axioms (notably efficiency / local accuracy), and the same framework yields both local explanations and robust global interpretations; efficient estimators exist (KernelSHAP, TreeSHAP).

Theory Reference

Chapter – Post-Modeling Explainability, Sections **Shapley Values for XAI** (Game Theory Origins, Properties of the Shapley Value) and **SHAP - 2017** (The Additive Feature Attribution Model, Properties of Additive Feature Attributions, Global Insights from SHAP, Advantages of SHAP).

5.16 Gradient — What the Gradients Are Computed With Respect To — June 2026 (Exam) · 1 pt · MCQ

Task

In **gradient-based explainability**, gradients are computed with respect to:

- ☐ The training set size.
- ☐ The number of classes only.
- ☐ The optimizer learning rate.
- ☐ The model input features.

Solution

Solution

The model input features.

Gradient-based attribution methods compute the gradient of the model **output** (e.g. the class score) with respect to the **input features** (e.g. the pixels of an image).
This measures how sensitive the prediction is to a small change in each input feature, producing a saliency/attribution map over the input.

Analysis of Options

- *The training set size.* — **Wrong:**
  The dataset size is a data property, not a differentiable quantity used for attribution.
- *The number of classes only.* — **Wrong:**
  The number of classes is fixed; the gradient is taken of a chosen output score w.r.t. the input, not of the class count.
- *The optimizer learning rate.* — **Wrong:**
  The learning rate is a *training* hyperparameter; explanation gradients are computed at inference on a trained model.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Gradient-based Explainability Methods** (*Vanilla Gradient / Saliency Map*):

Gradient-based methods attribute importance by differentiating the output score with respect to the input features, quantifying the local sensitivity of the prediction to each input.

5.17 Gradient — Key Idea of Integrated Gradients

— May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What is the key idea behind **Integrated Gradients**?

- ☐ Compute gradients at a single point and normalize them using feature statistics.
- ☐ Accumulate gradients along a path from a baseline input to the actual input.
- ☐ Replace gradients with feature importance learned from a surrogate model.
- ☐ Average gradients across multiple models trained with different initializations.

Solution

Solution

Accumulate gradients along a path from a baseline input to the actual input.

Integrated Gradients interpolates a path between a **baseline** x' ("no information", e.g. a black image or zero vector) and the input x, takes the gradient at every interpolated point, averages them, and scales by $(x_i - x'_i)$.
This accumulation solves the *gradient saturation* problem that makes plain gradients (and Gradient $\times$ Input) fail the Sensitivity axiom.

Analysis of Options

- *Gradients at a single point, normalized.* — **Wrong:**
  That describes Vanilla Gradient; the defining feature of IG is integration along a path.
- *Replace gradients with surrogate importance.* — **Wrong:**
  IG is a gradient-based method; it does not use a surrogate model.
- *Average gradients across multiple models.* — **Wrong:**
  Averaging over *noisy inputs* is SmoothGrad; IG integrates along the baseline-to-input path of a single model.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Integrated Gradients** (*The Integrated Gradients Approach*, *Mathematical Formulation*):

Integrated Gradients attributes importance by integrating gradients along a path from a baseline to the input, satisfying the Sensitivity and Implementation-Invariance axioms.

5.18 Gradient — Main Limitation of Grad-CAM –

May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What is the main limitation of **Grad-CAM** compared to pixel-level attribution methods?

- ☐ It cannot be applied to convolutional neural networks trained on image data.
- ☐ It highlights broad regions rather than providing fine-grained feature importance.
- ☐ It requires access to the full training dataset to compute explanations.
- ☐ It cannot distinguish between positive and negative feature contributions.

Solution

Solution

It highlights broad regions rather than providing fine-grained feature importance.

Grad-CAM operates on the **last convolutional feature maps**, which have low spatial resolution.
The resulting heatmap is therefore a **coarse localization map**, localizing regions but not individual pixels — which is why Guided Grad-CAM combines it with a pixel-level method.

Analysis of Options

- *Cannot be applied to CNNs.* — **Wrong:**
  Grad-CAM is designed *specifically* for CNNs.
- *Requires the full training dataset.* — **Wrong:**
  It needs only a forward and backward pass on the single input, not the training set.
- *Cannot distinguish positive from negative contributions.* — **Wrong:**
  Grad-CAM applies a ReLU to keep *positive* influences and is class-discriminative; its real weakness is spatial coarseness.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Grad-CAM** (*Mathematical Process*, *Guided Grad-CAM*):

Grad-CAM produces a coarse, class-discriminative localization map from the last convolutional layer; it is often combined with pixel-level methods (Guided Grad-CAM) for finer detail.

5.19 Gradient — Grad-CAM vs. Vanilla Gradient

— May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What distinguishes **Grad-CAM** from **Vanilla Gradient** in terms of where the gradient is propagated?

- ☐ Grad-CAM propagates the gradient to the input image; Vanilla Gradient stops at the last conv layer.
- ☐ Grad-CAM propagates to the last convolutional layer; Vanilla Gradient goes all the way to the input.
- ☐ Both propagate to exactly the same layer.
- ☐ Grad-CAM only uses forward pass information; no backpropagation is needed.

Solution

Solution

Grad-CAM propagates to the last convolutional layer; Vanilla Gradient goes all the way to the input.

Grad-CAM computes gradients of the class score with respect to the **last convolutional feature maps** A^k (the gradient is *not* backpropagated to the image), while Vanilla Gradient backpropagates all the way to the **input pixels**.
This is exactly why Grad-CAM is coarse and Vanilla Gradient is pixel-level.

Analysis of Options

- *Grad-CAM to input; Vanilla stops at conv.* — **Wrong:**
  This reverses the two methods.
- *Both propagate to the same layer.* — **Wrong:**
  They target different layers, which is the whole distinction.
- *Grad-CAM needs no backpropagation.* — **Wrong:**
  Grad-CAM explicitly uses gradients (a backward pass) to compute the weights α_k^c of the feature maps.

Theory Reference

Chapter – Post-Modeling Explainability, Sections **Vanilla Gradient** (gradient w.r.t. the input) and **Grad-CAM** (*Intuition*: gradient into the last convolutional layer, not backpropagated to the image).

5.20 Gradient — The Sensitivity Axiom — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Sensitivity is a key axiom for attribution methods. Which statement best defines it?

- ☐ Models with identical input-output behavior should produce identical attribution scores.
- ☐ If two inputs differ in one feature and yield different outputs, that feature must have non-zero attribution.
- ☐ The total attribution across features should equal the difference between prediction and baseline.

Solution

Solution

If two inputs differ in one feature and yield different outputs, that feature must have non-zero attribution.

The **Sensitivity** axiom requires that whenever changing a single feature changes the prediction, that feature receives a **non-zero** attribution.
Plain gradients (and Gradient × Input) violate this under saturation, which Integrated Gradients fixes.

Analysis of Options

- *Identical behavior ⇒ identical attributions.* — **Wrong:**
  That defines *Implementation Invariance*, the other axiom proposed alongside Integrated Gradients.

- *Attributions sum to prediction minus baseline.* — **Wrong:**
  That describes *Completeness* (the efficiency-style property), not Sensitivity.

Theory Reference

Chapter – Post-Modeling Explainability, Section **Integrated Gradients** (axioms: *Sensitivity* and *Implementation Invariance*; *Gradient × Input Fails Sensitivity*):

Sensitivity: if an input and baseline differ in one feature and produce different predictions, that feature must receive non-zero attribution.

5.21 Explaining a Black-Box Healthcare System (Tabular Data) — May 2025 (Sample) · 7 pts · Open

Task

Consider a trained black-box healthcare AI system operating with *tabular* data that predicts the likelihood of disease. How would you ensure that this system provides understandable explanations to doctors? Discuss some methods and the types of explanations that would be appropriate.

Solution

Solution

Healthcare AI System is **already trained** and is a **black-box**, so I need to use **Post-hoc** Explainable method.
Because the data is **tabular**, the raw features are already an *interpretable representation*, so we can explain directly on them.

Global Methods

For the doctors to trust the *overall* system, they need to check that it behaves in a clinically and plausible way:

- **Surrogate Model**: *interpretable* proxy trained to approximate the prediction function of the Healthcare AI System, by using a completely new surrogate prediction function to obtain a simplified and understandable representation.
  Surrogate Model *must* be *inherently interpretable*.
  I would discard this option considering that it would just be an *approximation* of the complex Healthcare AI System.

- **Permutation Feature Importance (PFI)**: this method provides **attribution-based** explanations, ranking the features from the *most* important to the least one.
  To compute the importance of a feature, it *randomly* shuffles its values checking the drop in the performance.
  Higher is the drop in the performance, higher is the importance of that feature.

- **Partial Dependence Plots (PDPs)**: this method is a visualizatino tool used to understand the relationship between a prediction and a subset of features, revealing *how* changes in features influence the prediction.
  This method provides **visualization-based** explanations that are human-understandable.

Local Methods

After investigating *globally* the AI System, doctors need to investigate the *specific* patient's prediction.

- **Surrogate Model (LIME)**: *interpretable* proxy trained to approximate the prediction function of the Healthcare AI System, *within* the *neighborhood* of a prediction.
  Approximate the black-box Model with a *surrogate* Model (always *inherently interpretable*), restricted only to the *local proximity* of the instance being explained.
  In particular, **LIME** uses a Linear Model as *surrogate* and uses the *perturbed* samples to generate the neighborhood, weighting the samples by-proximity (**feature attributions)** to the instance being explained.
  I would discard this method considering the *variance* in explanations (not the best option in medical diagnosis) due to this *random* sampling.

- **SHAP**: this method is based on the **Shapley Values** that's concept borrowed from Game Theory.
  Each player (feature) contributes with a certain contribute (Shapley Value) to a common objective (prediction) within a coalition of players (subset of features).
  For computing the **Shapley Values**, we need to average the marginal contributions across all subsets of features.
  SHAP is based on the **Shapley Values** and uses a Linear Model based on **Additive Feature Attributions** to approximate the **Shapley Values** of the features that are simply represented by weights of the Linear Model.
  It allows to consider the *interactions* between features, that's very important for clinical diagnosis.

- **Counterfactuals**: this method, changing *some aspects* of the input to check how the output changes, allows to understand better the *decision boundary* of the AI System and *why* a prediction was made for a *specific* patient.
  Actionability is a property of **Counterfactuals** that provides insights on *how* doctors can change the outcome (predicted desease for a patient), applying modifications in clinical features.

My Choice

I would choose an **hybrid solution**:

1. **Global methods**: doctors can understand the *overall* behavior of the AI System, before trusting it.
   I would prefer to use **PDPs** because they offer **visualization-based explanations** that can be *easily* understood by doctors.

2. **Local methods**: doctors can understand and justify *each* patient's prediction.
   I would prefer to use **SHAP** because of the **importance score** and the consideration of *interactions*.
   Also **Counterfactuals** could be an option because of **Actionability**, very useful for doctors to understand and modify the clinical features for the patients.

5.22 Explaining a Black-Box Medical-Imaging Diagnosis System — June 2025 (Exam) · 6 pts · Open

Task

A healthcare provider uses an AI system to diagnose diseases from medical images (e.g. X-rays or MRIs). It is important that the AI provides understandable explanations to doctors. Describe at least two explainability methods that could help doctors understand the system's decisions. Discuss the types of explanation representation (e.g. feature importance, local rules, visualization, example-based) that would be most appropriate for this scenario and why. Choose one explainability method you would recommend, and justify your choice based on the needs of healthcare professionals.

Solution

Solution

The Diagnosis System is **already trained** and is a **black-box**, so I need to use **Post-hoc** Explainable methods.
Because the data is made of **images** (*not* tabular), the raw pixels are *not* an interpretable representation on their own: **visualization-based** (saliency maps) and **example-based** representations are more appropriate than a raw per-pixel feature importance.

Methods

Doctors (radiologists) already reason about image *regions*, so the explanation should tell them *where* the model is looking and *whether* it is clinically plausible.

- **Gradient-based Saliency Maps**: this method computes the *gradients* of the output with respect to the input to quantify *which* parts of the image are most influential for the prediction, producing a **visualization-based** explanation overlaid as a heatmap.
  - **Vanilla Gradient**: features with *larger* gradient mean that *small* changes there cause *significant* changes in the prediction.
    I would discard this option because *small* changes in the input might produce *noisy* Saliency Maps.
  - **SmoothGrad**: adds *Random Gaussian Noise* generating different versions of the input, computes the gradients on *all* the noisy points and *averages* them to remove the noise, giving cleaner maps.
  - **Grad-CAM**: designed for **CNNs**, uses the gradient information of the *last* convolutional layer to assign importance to each neuron.
    It highlights *broad* class-discriminative regions rather than fine-grained per-pixel importance, and requires only a single forward/backward pass.
  - **Integrated Gradients**: accumulates the gradients along a *path* from a baseline input to the actual input, satisfying **Sensitivity** and **Implementation Invariance**, so it fixes the gradient-saturation problem and gives finer per-pixel detail.
- **Example-based Explanations**: this method explains a prediction by *comparing* it with known cases.
  - **Prototypes**: show representative images with the *same* diagnosis.
  - **Case-Based Reasoning**: retrieve *similar* past patient images and their *confirmed* outcomes.

Most Appropriate Representations and Why

- **Visualization-based** (heatmaps / saliency maps): *intuitive* for radiologists, who already reason about image regions, and let them verify that the model is attending to *clinically relevant* areas.
- **Example-based**: aligns with the clinical practice of *comparing* against known cases, building **trust**.

Note on image data. If a removal/perturbation method such as **LIME** or **SHAP** is used, it *cannot* work on raw pixels: it must operate on an **interpretable representation** of the image (e.g. **super-pixels** toggled by a binary vector) together with the mapping h_x back to the original image. This is why I do *not* recommend a raw feature-importance representation here.

My Choice

I would recommend a **Grad-CAM-style visualization** (optionally **Guided Grad-CAM** or **Integrated Gradients** for finer detail):

1. It answers *what* and *where* the model is attending to, in a form that doctors can **validate** against their medical knowledge.
2. It produces a **visualization-based** explanation (heatmap), which matches how radiologists already read images, unlike a per-pixel importance list.
3. It is *efficient*, requiring only a single forward/backward pass, so it fits a real clinical workflow.

I would combine it with **Example-based** explanations (Prototypes / Case-Based Reasoning) to further build trust, since doctors are used to reasoning by comparison with previous confirmed cases.

5.23 Explaining a Fraud-Detection System (Tabular Data) — June 2026 (Exam) · 5 pts · Open

Task

A financial institution uses an AI system to detect potentially fraudulent credit-card transactions. The model operates on *tabular* data (transaction amount, merchant category, location, time of day, customer history, recent spending patterns). Since incorrect predictions may block legitimate transactions or let fraud pass undetected, the institution wants the system to provide understandable explanations to fraud analysts.

Describe at least **three** explainability methods that could be used to help analysts understand the system's decisions. Discuss the types of explanation representation that would be most appropriate for this scenario (e.g. feature importance, local rules). Choose **one** method you would recommend and clearly justify your choice based on the needs of fraud analysts.

Solution

Solution

The Fraud-Detection System is **already deployed** and is a **black-box**, so I need to use **Post-hoc**, **Model-agnostic** Explainable methods.
Because the data is **tabular**, the raw features are already an *interpretable representation*, so we can explain directly on them: **feature attribution** and **local rules / counterfactuals** are the most suitable representations.
Since analysts need to justify *each* flagged transaction, I focus primarily on **Local** methods.

Methods

Analysts need to understand *why* a *specific* transaction was flagged and be able to *justify* it (auditability).

- **Surrogate Model (LIME)**: *interpretable* proxy trained to approximate the prediction function of the Fraud-Detection System, *within* the *neighborhood* of a prediction.
  LIME uses a Linear Model as *surrogate*, generating the neighborhood with *perturbed* samples weighted by-proximity, and the surrogate weights are the **feature attributions**.
  I would be cautious with this method considering the *variance* in explanations due to the *random* sampling (analysts need stable justifications).

- **SHAP**: this method is based on the **Shapley Values**, a concept borrowed from Game Theory.
  Each player (feature) contributes with a certain contribute (Shapley Value) to a common objective (prediction) within a coalition of players (subset of features).
  Computing the exact **Shapley Values** is *exponential* (2^n coalitions), so **SHAP** approximates them via an **Additive Feature Attribution** Linear Model, satisfying properties like *efficiency*, *symmetry*, *null player*.
  It provides *signed* per-feature contributions (bar/force plots) and considers *interactions* between features.

- **LORE**: like LIME but fits a **Decision Tree** on a neighborhood built with a **genetic algorithm**, yielding a **local decision rule** (why this transaction was flagged) *and* **counterfactual rules** (what minimal change would flip the decision).

- **Counterfactuals**: this method presents the *smallest change* to the transaction's features that would flip the prediction.
  Actionability is a property of **Counterfactuals** that tells *how* the outcome could change, and it helps analysts understand the *decision boundary* of the System.

Most Appropriate Representations and Why

- **Local feature importance** (SHAP / LIME): tells the analyst *which* attributes (e.g. unusual amount, foreign location, odd time) drove the "fraud" score for *this specific* transaction.
- **Local rules / counterfactuals** (LORE): give crisp, *auditable* "if–then" reasoning and a *contrastive* "what would have been non-fraud", well suited to analysts who must **justify** blocking a customer.

My Choice

I would recommend **LORE**:

1. Its **decision-rule** representation maps directly onto how analysts *reason* and can be **audited** and documented, which is essential in a financial setting.
2. Its **counterfactual rules** indicate what would make the transaction *legitimate* (**actionability**).
3. It is computationally *lighter* than the exact Shapley computation.

As an alternative, **SHAP** is an equally strong choice for its *theoretical guarantees* (consistency, efficiency) and its *stable, signed* contributions, which are more reliable than LIME's random-sampling variance.

Theory Reference

Chapter – Post-Modeling Explainability, Sections **Local Surrogate Interpretable Models** (LIME, LORE), **Shapley Values for XAI / SHAP** (game-theory origin, coalitions, additive feature attribution, axioms), and **Counterfactual Explanations** (cross-reference). Feature importance vs. local rules relate to the *Explanation Representation* taxonomy.

Chapter 6

Evaluating Explainability

6.1 Faithfulness vs. Plausibility — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What is the key difference between **faithfulness** and **plausibility**?

- ☐ Faithfulness is about humans, plausibility about models.
- ☐ Faithfulness is about model behavior, plausibility about human reasoning.
- ☐ They are equivalent.
- ☐ Plausibility is quantitative, faithfulness is qualitative.

Solution

Solution

Faithfulness is about model behavior, plausibility about human reasoning.

Faithfulness measures how well the explanation aligns with the actual model's behavior and inner workings — it answers *"Is this exactly how the model made its decision?"*
Plausibility measures the alignment of the explanation with human reasoning — it answers *"Is this what we expect as humans?"* There is no guarantee that a plausible explanation is faithful, or vice versa.

Analysis of Options

- *Faithfulness is about humans, plausibility about models.* — **Wrong:** This swaps the two definitions.
- *They are equivalent.* — **Wrong:** They are distinct and can diverge — a highly plausible explanation need not reflect the model's inner reasoning.
- *Plausibility is quantitative, faithfulness is qualitative.* — **Wrong:** The distinction is about *what* is assessed (model behavior vs. human reasoning), not quantitative vs. qualitative.

Common Mistake

We cannot assume explanations are faithful by default — they must be tested, as explainers can fail. A *non-plausible* explanation means either the model reasoned incorrectly, or the model reasoned correctly but the explainer failed to capture it.

Theory Reference

Chapter – Evaluation of Explanations, Section **Plausibility vs. Faithfulness**:

Plausibility is the alignment of the explanation with human reasoning; faithfulness is the alignment of the explanation with the actual model's behavior and inner workings. The two are independent and must not be conflated.

6.2 Faithfulness vs. Plausibility — Can an Explanation Be Plausible but Not Faithful? — June 2026

(Exam) · 1.5 pts · Open

Task

Explain the difference between **faithfulness** and **plausibility** in the evaluation of explanations. Can an explanation be *plausible but not faithful*? Motivate your answer.

Solution

Solution

Plausibility = alignment of the explanation with **human reasoning**: "Is this what we, as humans, expect?"
Faithfulness = alignment of the explanation with the **model's actual inner workings / decision process**: "Is this really how the model made its decision?"
They are **independent**: an explanation can look convincing to a human yet not reflect the true computation of the model.

Can It Be Plausible but Not Faithful? — Yes

- There is **no guarantee** that a highly plausible explanation reflects the model's real reasoning, and vice versa. An explanation may match human intuition (**plausible**) while being disconnected from what the model actually computed (**not faithful**).
- **Concrete case:** a model may reach the right answer *for the wrong reason* (e.g. relying on a spurious background/shortcut feature). An explainer — or a model trained to imitate human rationales (as in **teaching-to-explain / TED**-style setups) — can output a human-pleasing justification that hides this, so the explanation is plausible but unfaithful.
- Conversely, a **non-plausible** explanation does not necessarily mean the explainer failed: either the model truly reasoned in an odd (but real) way, *or* the model reasoned well and the explainer failed to capture it.

This is exactly why faithfulness **cannot be assumed** and must be **explicitly tested** (removal-based tests, sanity checks, white-box / synthetic-data checks), rather than judged by how "reasonable" the explanation looks to a human.

Common Mistake

Do *not* define faithfulness merely as "the model's ability to solve the task" — that is model *accuracy*. **Faithfulness** is a property of the *explanation*: how truthfully it reflects the model's decision process.

Theory Reference

Chapter – Evaluating Explainability, Section **Plausibility vs. Faithfulness**:

Plausibility measures agreement with human rationales; faithfulness measures agreement with the model's true reasoning. A highly plausible explanation is not guaranteed to be faithful, so faithfulness must be tested (removal-based evaluation, sanity checks).

6.3 Removal-based Evaluation of Top-k Features —

May 2026 (Interactive Questions) · 1 pt · MCQ

Task

In removal-based evaluation, what indicates a good explanation when removing the top-k important features?

- ☐ The model output does not change.
- ☐ The model output slightly increases.
- ☐ The model output significantly decreases.
- ☐ The explanation becomes shorter.

Solution

Solution

The model output significantly decreases.

This is the **Incremental Deletion** test of **Comprehensiveness (Correctness)**: if the features flagged as most important truly drive the prediction, removing the top-k of them should cause a **high drop** in the model's confidence.
The higher the drop, the better the comprehensiveness — it confirms the explanation captured the genuinely influential features.

Analysis of Options

- *The output does not change.* — **Wrong:**
  No change would mean the "important" features were not actually important — a sign of an *unfaithful* explanation.
- *The output slightly increases.* — **Wrong:**
  Removing genuinely supportive features should reduce, not raise, confidence in the class.
- *The explanation becomes shorter.* — **Wrong:**
  Explanation length relates to *compactness*, not to the deletion test's measure of faithfulness.

Theory Reference

Chapter – Evaluation of Explanations, Section **Methods for Evaluating → Evaluating Faithfulness (A. Removal-based)**, *Example 1: Evaluating Comprehensiveness via Incremental Deletion*:

After filtering out negatively-contributing attributes, progressively remove the k most important attributes (k from 10% to 100%) and average the probability drops; the higher the drop, the better the comprehensiveness. (Summarized via metrics such as the Area over the Perturbation Curve.)

6.4 Capturing Completeness (Sufficiency) — May 2026

(Interactive Questions) · 1 pt · MCQ

Task

Which evaluation setup best captures **completeness** (sufficiency) of an explanation?

- ☐ Removing the most important features and measuring the drop in prediction confidence.
- ☐ Keeping only the most important features and measuring if the prediction remains similar.
- ☐ Comparing explanations across models that produce identical outputs for all inputs.
- ☐ Measuring how similar explanations are for slightly perturbed versions of the same input.

Solution

Solution

Keeping only the most important features and measuring if the prediction remains similar.

Completeness (Sufficiency) asks whether the highlighted elements are *enough* on their own to justify the prediction.
In the theory's Incremental Deletion procedure, this is tested by removing the *least* important attributes (keeping only the important ones) and checking that the output is essentially preserved: the closer the drop is to 0, the better the sufficiency.

Analysis of Options

- *Removing the most important features and measuring the drop.* — **Wrong:** That is the Comprehensiveness (Correctness) test, which probes whether *all* relevant features were captured — not sufficiency.
- *Comparing across models with identical outputs.* — **Wrong:** That relates to *Consistency / Implementation Invariance*, not completeness.
- *Similarity under small input perturbations.* — **Wrong:** That measures *Continuity (stability)*, a different criterion.

Theory Reference

Chapter – Evaluation of Explanations, Section **Methods for Evaluating → Evaluating Faithfulness (A. Removal-based)**, *Example 2: Evaluating Sufficiency via Incremental Deletion*:

After filtering out negatively-contributing attributes, progressively remove the k *least* important attributes (keeping the important ones) and average the drop; a drop close to 0 indicates the highlighted features are sufficient to justify the prediction.

6.5 Detecting an Unfaithful Explanation — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Which scenario best indicates that an explanation method is **not faithful**?

- ☐ The explanation highlights features that align well with human intuition across many samples.
- ☐ The explanation remains almost unchanged after randomizing the model parameters.
- ☐ The explanation varies smoothly when the input is slightly perturbed with small noise.
- ☐ The explanation uses a small number of features and avoids redundant information.

Solution

Solution

The explanation remains almost unchanged after randomizing the model parameters.

A faithful explanation must be sensitive to the model's **learned parameters**. In the *Sanity Check*, we compare the explanation for the trained model f against the explanation after completely randomizing / re-initializing its weights — we **expect a massive change**. If there is no change, the explanation is insensitive to f: it relies entirely on the input and does not reflect the model's reasoning, hence it is **unfaithful**.

Analysis of Options

- *Aligns with human intuition.* — **Wrong:**
  That indicates *plausibility*, which says nothing about faithfulness.
- *Varies smoothly under small input noise.* — **Wrong:**
  Smooth variation indicates good *continuity/stability*, a desirable property, not a failure.
- *Uses few, non-redundant features.* — **Wrong:**
  That describes *compactness* (size, redundancy), which is unrelated to whether the explanation tracks the model.

Common Mistake

The *parameter randomization sanity check* is a standard faithfulness test: an explanation that survives weight randomization is insensitive to the model and therefore unfaithful.

Theory Reference

Chapter – Evaluation of Explanations, Section **Methods for Evaluating → Evaluating Faithfulness (B. Sanity Checks)**:

Compare the explanation of the trained f with that of a randomized/re-initialized model; we expect a massive change. No change means the explanation is insensitive to f and relies entirely on the input.

6.6 Testing Contrastivity (Target Sensitivity) — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Which evaluation best tests **contrastivity** (target sensitivity)?

- ☐ Comparing explanations for the same input when the target class is changed.
- ☐ Comparing explanations for similar inputs that differ by a small perturbation.
- ☐ Measuring the drop in prediction when removing the most important features.
- ☐ Measuring the agreement between the explanation and a surrogate model prediction.

Solution

Solution

Comparing explanations for the same input when the target class is changed.

Contrastivity describes how discriminative an explanation is with respect to other targets: it should explain the *"why"* and the *"why not"*.
Target Sensitivity is tested by computing the similarity between explanations generated for the **same input** x **but targeting different output classes**: the larger the difference (lower similarity), the better the contrastivity.

Analysis of Options

- *Similar inputs differing by a small perturbation.* — **Wrong:**
  That tests *Continuity (stability)*, not target sensitivity.
- *Drop in prediction when removing features.* — **Wrong:**
  That is the removal-based *faithfulness* (comprehensiveness) test.
- *Agreement with a surrogate model.* — **Wrong:**
  That is closer to the *Fidelity* check, unrelated to varying the target class.

Theory Reference

Chapter – Evaluation of Explanations, Section **Methods for Evaluating → Evaluating Contrastivity (Target Sensitivity)**; relies on the **Separability Property** from *Properties of Explanation Quality → Contrastivity*:

Compute the similarity between explanations for the same input x across different target classes; non-identical instances of different classes must have dissimilar explanations, so lower similarity means better contrastivity.

Chapter 7

Concept-based Explainable AI

7.1 Limitations of Saliency Maps and the Concept-based Remedy — June 2025 (Exam) · 3 pts · Open

Task

What are the main limitations of standard explanation methods (like saliency maps)? How do **concept-based** explainability methods try to solve them?

Solution

Solution

Standard feature-attribution / saliency-map methods point at *low-level pixels* rather than the human-understandable *concept* the network identified. **Concept-based XAI** (C-XAI) raises the level of abstraction, explaining predictions through high-level, human-understandable concepts (colors, objects, abstract ideas) instead of raw input features (pixels or text tokens).

Main Limitations of Saliency Maps

- **Ambiguity of saliency maps:** it is hard to judge which explanation method is better just by looking at the maps; saliency maps for completely different classes can look very similar, so it is unclear which class is actually being explained.
- **Similarity to edge detectors:** methods that heavily weight the input values (e.g. Gradient $\times$ Input) produce explanations almost identical to standard edge detectors.
- **Input-dependence over model-dependence:** randomizing the weights of a few layers barely changes the explanation — even a fully randomized network can yield a similar map — showing the method depends more on the input image than on the actual learned weights of the model.

How C-XAI Addresses Them

By interpreting and evaluating the model using high-level, human-understandable *concepts* rather than individual pixels/tokens, C-XAI *bridges the cognitive gap* between the model's internal representations and human reasoning. It shifts the focus from low-level feature attribution to higher-level abstractions, answering not just *where* the model is looking, but *what* the model is actually seeing.

Concretely, C-XAI realizes this through its pipeline (data projection $\rightarrow$ concept annotation $\rightarrow$ concept representation/alignment $\rightarrow$ concept-based explanation) and can be **post-hoc** (e.g. T-CAV, Network Dissection, ACE) or **explainable-by-design** (e.g. Concept Bottleneck Models), producing explanations as class–concept relations, node–concept associations, or concept visualizations.

Theory Reference

Chapter – Concept-based Explainable AI, Section **Motivation** (Ambiguity of Saliency Maps, Similarity to Edge Detectors, Input-Dependence over Model-Dependence); remedy grounded in the **C-XAI** definition (high-level human-understandable concepts; *what* vs. *where*), with realization via the **C-XAI Pipeline**, **Post-hoc vs. Explainable-by-Design**, and **Types of Concept-based Explanations**.

7.2 Explainable-by-Design with Concepts but No Concept Labels — June 2026 (Exam) · 3 pts · Open

Task

You want to create an **explainable-by-design** model that uses **concepts** as explanations. However, you **do not have concept labels** on the training dataset. Describe at least a few approaches that can be employed in this scenario.

Solution

Solution

An **explainable-by-design** (in-modeling) model is trained to justify its predictions through **human-understandable concepts**. Standard Concept Bottleneck Models require **concept annotations**, which are expensive. Without concept labels, we use **label-free** approaches that either *discover* concepts unsupervised or *generate* them from an external source.

Approaches (no concept labels required)

- **Unsupervised — Prototype-based** *(e.g. ProtoPNet):* the network is trained (using only class labels) to learn a set of representative **prototypes** (patches of training examples) in its latent space. At inference it classifies by **similarity** to these prototypes, and the explanation shows which part of the input most activates each prototype ("this looks like that").
- **Unsupervised — Concept Basis** *(e.g. SENN):* the model jointly learns a set of unsupervised **concepts** (basis) and predicts as an interpretable combination of them. Concepts are defined by cluster centroids in latent space and visualized via nearest samples or an auto-encoder decoder.
- **Hybrid supervised/unsupervised extra-capacity** *(e.g. concept-whitening / hybrid CBMs):* some neurons are reserved to freely learn an **unsupervised, clustered** representation to capture information missing from any (partial) supervision.
- **LLM / Generative concept generation:** query an external **generative model (LLM)** for a textual description of each class (e.g. "What is a parrot?"), decompose it into **textual concepts** ("curved beak", "bright feathers"), and align their embeddings with the input's latent representation — **eliminating the need for human concept labeling** entirely.

Mechanism differences matter (name and briefly describe them): prototype-based methods explain by *similarity to learned examples*; concept-basis methods explain via a *learned interpretable basis*; LLM-based methods *import* concepts as text.

Common Mistake

State the trade-offs: purely **unsupervised** concepts are *not always human-interpretable* and often need concept visualization to be understood; **LLM-generated** concepts remove labeling cost but rely on the external model having accurate, unbiased domain knowledge (and still need per-class querying).

Theory Reference

Chapter – Concept-based Explainable AI, Sections **Types of Concept-based Explanations** (Unsupervised Concept Bases, Prototype Concepts, Generative/LLM concepts) and **Explainable-by-Design Models**: **Unsupervised (Prototype-based) – ProtoPNet**, **Unsupervised (Concept Basis) – SENN**, hybrid supervised/unsupervised, and **Generative C-XAI (LLM-based)**, none of which require concept annotations.

Chapter 8

Introduction to Natural Language Processing

Unfortunately, no questions published for this Chapter until now!

Chapter 9

Counterfactual Explanations

9.1 Counterfactual Explanations for Model Predictions — May 2025 (Sample) · 4 pts · Open

Task

Discuss how **counterfactual explanations** can be used to provide insights into model predictions. Provide an example scenario where counterfactual explanations would be useful.

Solution

Solution

A **counterfactual explanation** answers the question *"What if...?"*: it describes the **smallest change to the feature values that changes the prediction to a predefined output** $y' \neq y$.
It is a *local* explanation and an *example-based* one, since the counterfactual x' is itself a new instance derived from x with a few features changed.

Insights Provided (Why Counterfactuals?)

- **Interpretability:** they help users understand the model's *decision boundary* near the instance and exactly why a prediction was made, while remaining simple (only a few features change).
- **Trust:** they show explicitly how a decision could be altered, and reveal when a user should *contest* a decision (e.g. if flipping the outcome would require changing a sensitive/protected attribute).
- **Actionability:** they offer practical guidance on how an individual can change their real-world outcome.

Example Scenarios

- **Loan application rejection:** an applicant (Age 30, Income 30K, Amount 15K) is rejected; the counterfactual "request 12K instead of 15K" would lead to approval — concrete, actionable guidance.
- **Employee performance evaluation:** show an employee a minimal, achievable change (e.g. complete additional training) that would improve their rating.

Theory Reference

Chapter – Counterfactual Explanations, Sections **Introduction to Counterfactual Explanations**, **Formal Definition**, and **Why Counterfactual Explanations?** (Interpretability, Trust, Actionability).

9.2 Why Counterfactuals Are Example-Based — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Why are counterfactual explanations considered **example-based**?

- ☐ They rely on training dataset sampling.
- ☐ They produce new instances as explanations.
- ☐ They summarize feature importance globally.
- ☐ They approximate gradients of the model.

Solution

Solution

They produce new instances as explanations.

By the formal definition, a counterfactual is a **new instance** x' obtained from the original x by changing some of its feature values (e.g. "the same applicant but with the amount requested lowered to \$12K").
Because the explanation *is* an instance, counterfactuals belong to the family of **example-based** explanations.

Analysis of Options

- *They rely on training dataset sampling.* — **Wrong:**
  Counterfactuals are typically *generated* (e.g. by optimizing a loss), not necessarily drawn from the training set.
- *They summarize feature importance globally.* — **Wrong:**
  That describes global feature-attribution methods; counterfactuals are local and instance-shaped.
- *They approximate gradients of the model.* — **Wrong:**
  Gradients may be used during the search, but that is not why counterfactuals are example-based.

Theory Reference

Chapter – Counterfactual Explanations, Section **Formal Definition**:

A counterfactual is considered an example-based explanation because it represents a new instance x' that, starting from the original x, has some of its features changed.

9.3 Property Ensuring Minimal Change — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

Which property ensures **minimal change** between the original and the counterfactual input?

- ☐ Diversity.
- ☐ Sparsity.
- ☐ Proximity.
- ☐ Validity.

Solution

Solution

Proximity.

This is the theory's **Closeness to the Input** property: the counterfactual's features should be as similar as possible to the original instance, i.e. the *total magnitude* of change in feature values is small.
It is measured as the mean feature-wise distance ($CF\text{-}proximity = \frac{1}{k}\sum_i dist(x'_i, x)$), so minimizing it yields the smallest change achieving the desired outcome.

Analysis of Options

- *Diversity.* — **Wrong:**
  Diversity asks for several *structurally different* counterfactuals, not minimal change.

- *Sparsity.* — **Wrong:**
  Sparsity limits the *number* of changed features, not the total magnitude of change (which is proximity).

- *Validity.* — **Wrong:**
  Validity requires the counterfactual to actually flip the prediction to the desired class y', unrelated to how small the change is.

Common Mistake

Do not confuse **proximity** (small *total* change in feature values, the "Closeness to the Input" property) with **sparsity** (few *features* changed). A counterfactual can be sparse but far, or close but dense.

Theory Reference

Chapter – Counterfactual Explanations, Section **Properties of Counterfactual Explanations** (*Closeness to the Input*); measured by the *Proximity* metric in **Evaluating Counterfactual Explanations**:

Closeness to the Input requires the counterfactual's features to be as similar as possible to the original instance; proximity is the mean feature-wise distance $\frac{1}{k}\sum_i dist(x'_i, x)$.

9.4 Meaning of Sparsity — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What does **sparsity** refer to in counterfactual explanations?

- ☐ Using fewer training samples.
- ☐ Changing only a few features.
- ☐ Reducing model complexity.
- ☐ Limiting output classes.

Solution

Solution

Changing only a few features.

Sparsity requires the counterfactual to change **only a few features**. This keeps the recommended change easy to understand and act upon (e.g. "request a lower amount" rather than altering many variables at once). It is measured by counting the changed features, and in DiCE it is applied as a *post-processing* step rather than a loss term.

Analysis of Options

- *Using fewer training samples.* — **Wrong:** Sparsity concerns the explanation, not the training data.
- *Reducing model complexity.* — **Wrong:** Sparsity is a property of the counterfactual, not of the model.
- *Limiting output classes.* — **Wrong:** The number of output classes is unrelated to how many features the counterfactual changes.

Theory Reference

Chapter – Counterfactual Explanations, Section **Properties of Counterfactual Explanations** (*Sparsity*); metric $CF\text{-}sparsity = \frac{1}{k}\sum_i \sum_l \mathbf{1}_{[x'_{i,l} \neq x_l]}$:

Sparsity favors counterfactuals that alter the smallest possible number of features, improving actionability and interpretability.

9.5 Least Actionable Counterfactual — June 2026 (Exam) · 1 pt · MCQ

Task

Which of the following counterfactuals is least actionable?

- ☐ Decrease your age by 10 years.
- ☐ Reduce the requested loan amount.
- ☐ Increase savings.
- ☐ Reduce current debt.

Solution

Solution

Decrease your age by 10 years.

A good counterfactual must be **actionable**: it should recommend changes the user can *realistically* act upon.
Age is an **immutable** feature — a person cannot change it — so this recommendation is impossible to act on, making it the least actionable.

Analysis of Options

- *Reduce the requested loan amount.* — **Actionable:**
  The applicant directly controls how much they request.
- *Increase savings.* — **Actionable:**
  Achievable over time through the user's behavior.
- *Reduce current debt.* — **Actionable:**
  The user can act to pay down debt.

Only *Decrease your age* targets an **immutable** / **non-actionable** feature.

Theory Reference

Chapter – Counterfactual Explanations, Section **Properties of Counterfactual Explanations** (*Actionability, Feasibility and Actionability*):

A counterfactual should only propose changes to *mutable, actionable* features; recommending changes to immutable attributes (age, race, place of birth) yields an unusable explanation.

9.6 Goal of the Loss in Wachter et al. — May 2026 (Interactive Questions) · 1 pt · MCQ

Task

What is the goal of the loss function in **Wachter et al.**'s approach?

- ☐ Minimize training loss while preserving model generalization.
- ☐ Balance closeness to the desired output and similarity to the input.
- ☐ Maximize diversity and sparsity of generated explanations.
- ☐ Reduce feature variance and normalize input distributions.

Solution

Solution

Balance closeness to the desired output and similarity to the input.

Wachter et al.'s loss $L = \lambda\,(f(x') - y')^2 + d(x, x')$ has two terms: the quadratic term enforces **Closeness to the Predefined Output** (prediction near the desired y'), and the distance term d enforces **Closeness to the Input**.
The parameter λ trades these off — larger λ favors hitting y', smaller λ favors staying close to x.

Analysis of Options

- *Minimize training loss while preserving generalization.* — **Wrong:**
  The model f is fixed; the counterfactual search optimizes x', it does not retrain the model.

- *Maximize diversity and sparsity.* — **Wrong:**
  Diversity is introduced later by DiCE; Wachter's loss targets only output-closeness vs. input-closeness.

- *Reduce feature variance and normalize input.* — **Wrong:**
  These are preprocessing notions; note that d *scales* feature differences by the median absolute deviation (MAD_k), but that is normalization *within* the distance term, not the loss's goal.

Theory Reference

Chapter – Counterfactual Explanations, Section **Counterfactual Generation Methods → Wachter et al. Approach** (*The Loss Function*):

$L(x, x', y', \lambda) = \lambda(f(x') - y')^2 + d(x, x')$ combines a term driving the prediction to the desired output with a distance term (d, scaled by MAD_k) keeping the counterfactual close to the original instance.

9.7 How DiCE Improves Counterfactual Generation

— May 2026 (Interactive Questions) · 1 pt · MCQ

Task

How does **DiCE** improve over basic counterfactual generation?

- ☐ Adds feature selection and normalization.
- ☐ Introduces diversity and feasibility constraints.
- ☐ Uses only linear models for explanations.
- ☐ Focuses only on classification problems.

Solution

Solution

Introduces diversity and feasibility constraints.

DiCE (Diverse Counterfactual Explanations) extends the Wachter approach to explicitly consider **Diversity** — generating a set of k structurally different counterfactuals, enforced via a Determinantal Point Process ($dpp_diversity = \det(K)$) that penalizes similar counterfactuals — and **Feasibility** — user-provided constraints on which features may change (e.g. income capped, age immutable).

Analysis of Options

- *Adds feature selection and normalization.* — **Wrong:**
  These are generic preprocessing steps, not the contribution of DiCE.
- *Uses only linear models.* — **Wrong:**
  DiCE is model-agnostic and does not restrict itself to linear models.
- *Focuses only on classification problems.* — **Wrong:**
  DiCE's defining idea is diverse, feasible counterfactuals, not a restriction to classification.

Theory Reference

Chapter – Counterfactual Explanations, Section **Counterfactual Generation Methods → DiCE** (*Terms in the Loss Function*: diversity via DPP; *Additional Constraints*: feasibility, with sparsity applied as post-processing):

DiCE produces a set of k diverse counterfactuals (DPP-based diversity term) subject to feasibility constraints, giving users several realistic alternative ways to change the outcome.

Chapter 10

Attention-based Explainability

Unfortunately, no questions published for this Chapter until now!

Chapter 11

Mechanistic Interpretability

11.1 False Statements about Mechanistic Interpretability — June 2025 (Exam) · 1 pt · MCQ

Task

Which of the following statements is **FALSE** regarding mechanistic interpretability?

- ☐ Mechanistic interpretability aims to reverse-engineer neural networks into human-understandable components like circuits and algorithms.
- ☐ Techniques in mechanistic interpretability often involve identifying specific neurons or patterns responsible for particular behaviors.
- ☐ All layers in a deep neural network encode the same types of interpretable features through mechanistic analysis.
- ☐ Mechanistic interpretability has shown promising results in understanding how models internally represent features like syntax or modular arithmetic.

Solution

Solution

All layers in a deep neural network encode the same types of interpretable features through mechanistic analysis. This is the **FALSE** statement. Different layers do *not* encode the same types of features. Features are organized hierarchically and composed into **circuits** — early layers detect low-level patterns (edges, textures), while deeper layers compose them into higher-level structures (object parts, pose-invariant detectors). Claiming all layers encode the same interpretable features contradicts the feature/circuit-composition view at the core of mechanistic interpretability.

Analysis of Options

- *All layers encode the same types of interpretable features.* — **FALSE (correct choice):**
  Mechanistic interpretability is built on the claim that NN representations are composed of individual **features** (edges, textures, object parts), each detected by a neuron or group of neurons, which are then connected by learned weights into **circuits** that perform increasingly complex computations across layers — not identically replicated in every layer. The pose-invariant dog head detector, assembled stage-by-stage from oriented fur detectors through to an orientation-invariant head, is a concrete illustration.

- *Reverse-engineer NNs into human-understandable components.* — **True:**
  Its central goal is to reverse-engineer the internal computational mechanisms (features, circuits, algorithms) of the entire model into globally understandable structures, rather than explaining a single prediction.

- *Identifying specific neurons or patterns responsible for behaviors.* — **True:**
  It works by *zooming in* on individual neurons and their connections/interactions to find the components responsible for specific behaviors (e.g. via feature visualization, dataset examples, synthetic testing).

- *Promising results on syntax or modular arithmetic.* — **True:**
  It has shown promising results, including in LLMs, e.g. understanding the internal representation of features such as syntax or modular arithmetic.

Common Mistake

Read "FALSE" carefully: the question asks for the statement that does *not* hold.
Here, three statements correctly describe mechanistic interpretability and only one is wrong.

Theory Reference

Chapter – Mechanistic Interpretability, Section **Mechanistic Interpretability in CNNs** → Subsection **The Three Speculative Claims** (*Features*, *Circuits*, *Universality*), and Section **Mechanistic Interpretability in LLMs**.

Mechanistic interpretability reverse-engineers a network's internal mechanisms into globally understandable structures by zooming into individual neurons and their interactions. Its three foundational claims are: **Features** (representations composed of individual learned patterns such as edges, textures and object parts, hierarchically organised across layers); **Circuits** (neurons connected by learned weights into structured computations, e.g. the multi-stage pose-invariant dog head detector); and **Universality** (similar features and circuits emerging across different architectures and tasks). Because features are arranged hierarchically and composed into circuits, layers encode *different* types of features — which is why the statement that all layers encode the same features is false.

Chapter 12

Adversarial Attacks and Defenses

12.1 Adversarial Attacks and Their Importance —

June 2025 (Exam) · 2.5 pts · Open

Task

What is an **adversarial attack**? Why are adversarial attacks important?

Solution

Solution

An **adversarial attack** is a **maliciously crafted example**, designed to make a neural network produce a *wrong* classification.
The perturbation is typically *imperceptible* to humans yet causes a confident misclassification.

Canonical Example

An image of a panda correctly classified with ∼60% confidence becomes, after adding a carefully crafted (visually imperceptible) adversarial perturbation, classified as a *gibbon* with ∼99% confidence.

Why They Are Important (one of the most critical open challenges in Trustworthy AI)

- They can be crafted to fool a **specific** network or **many** networks simultaneously, and they **transfer across networks** (an example crafted on network A often fools network B with different architecture, weights and independent training). This transferability enables **black-box attacks** via a surrogate model.
- A single perturbation may be valid for **many** input samples, not just one (universal perturbations).
- They are **not limited to image classification**: they affect generative models (autoencoder collapse), semantic segmentation, voice recognition, deep reinforcement learning and even LLMs (prompt injection, universal adversarial suffixes).
- They constitute a **concrete physical threat**: e.g. adversarial stickers on a stop sign make an autonomous car read "Speed Limit 45"; a 3-D printed turtle with adversarial texture is classified as a rifle from virtually any viewpoint, surviving the full rendering pipeline.
- We still do **not** know how to completely defend against them: most defenses are attack-specific and can be bypassed by new or modified attacks, creating an ongoing arms race.

Theory Reference

Chapter – Adversarial Attacks and Defenses, Section **Why are Adversarial Attacks Important?** and its subsections **Is the Threat Real?**, **Beyond Object Recognition**, **Transferability: Not Network Specific** and **Is There Any Effective Defense?**

Adversarial attacks are maliciously crafted examples designed to make a neural network provide a wrong classification, and represent one of the most critical open challenges in Trustworthy AI. Their importance rests on: the ability to fool a specific network or many networks at once; the existence of single perturbations valid for many samples; transferability across independently trained networks (enabling black-box attacks via surrogates); applicability across virtually every domain where neural networks are deployed; their realizability as physical objects effective across viewpoints, distances and lighting; and the fact that no current defense is fully robust.

12.2 Adversarial Attacks in Trustworthy AI — Are They a Real Threat? — June 2026 (Exam) · 2.5 pts · Open

Task

Why are **adversarial attacks** important in trustworthy AI? Are they a real threat to AI models? Motivate your answer with at least **two** examples or reasons.

Solution

Solution

An **adversarial attack** is a **maliciously crafted example** — typically an input with a perturbation that is **imperceptible** to humans (or a specific character sequence for text) — designed to make an AI system produce a **confidently wrong** output.
They matter because they directly threaten the **security, safety and reliability** of deployed models, and they are a **concrete real-world threat**, not merely theoretical.

Why They Are a Real Threat (at least two examples/reasons)

- **Physical road-sign attack (safety):** small stickers/patches placed on a **stop sign** (looking like graffiti to humans) can make an autonomous-driving perception system read it as a different sign (e.g. speed limit). The perturbation survives real-world **viewpoints, distances and lighting** — a direct physical-safety threat.
- **Security / evasion (e.g. weapon or object recognition):** an adversarial pattern printed on an object (e.g. a 3-D printed item consistently misread as a rifle, or a pattern that hides a weapon) lets malicious inputs **evade recognition systems**, defeating what the model was meant to protect.
- **Text-based attacks:** particular crafted character sequences / prompts can fool NLP and language models, extending the threat beyond images.
- **Transferability ⇒ black-box attacks:** an example crafted on one (surrogate) network often fools a *different*, independently trained network, so attackers do **not need access** to the target model.

Why Defense Is Hard

Defenses exist (e.g. adversarial training), but current methods have significant limitations: many are tailored to a **specific attack** and become ineffective once the attack changes, so there is an ongoing **arms race** with **no fully robust** defense — which is exactly why adversarial robustness is a central open challenge in Trustworthy AI.

Theory Reference

Chapter – Adversarial Attacks and Defenses, Sections **What Is an Adversarial Attack** (imperceptible perturbation, confident misclassification), **Real-World / Physical Threats** (Road Sign Attack, 3-D object attack), **Transferability** (black-box attacks via surrogate), and **Is There Any Effective Defense?** (defenses exist but are attack-specific / not fully robust).